Security & Cryptography

Franco Zambonelli
Computer Networks
Year: 2005

Security

Security Problems

- Privacy violation (see data, use services/Applications)
  - Of private nature, reserved to specific groups
  - On sale, not for free
- Destructive Attacks
  - Data destruction: delete HD content, consume memory or CPU
  - Denial of Service Attacks: cosume all resources (memory, CPU, disk) and make impossible their use

Security without Network

- Protect Data and Applications in Your PC
  - Lock the door!
- In a shared PC Workstation
  - Control and rule Accesses to File and Appls.
    - Several users with different usernames
    - A file has a “owner” – a specific users
    - Each file has “access control lists”, specifying who can do what on the file (franco can read and write, luca can olny read, andrea can neither read or write
    - Require similar access control lists for memory, HD space, CPU time
- Requires:
  - Identification, authentication, authorization

Terminology

- Identification: tell me who you are (username)
- Authentication: identifying someone (or something) reliably. *Proving you are who you say you are.* (password)
- Authorization: permission to access a resource. (access control list)

Examples: UNIX & NT

- UNIX
  - users can enter (log in) a system by specifying username and passwd
  - each file has an access control list specifying: who can read, write, or execute, a file
  - Access control lists specify how much disk space and CPU time a user can use
  - A special user (root) can do everything
- Win NT
  - users enter (log in) a system by specifying username and passwd
  - Each user is given a different perspective of the file system (see only a limited number of directories)
  - A special user (administrator) can do everything

Security in Networked Env.s

- Putting a host in the Internet means to: **Have always some unlocked door**
  - Because the hosts make services available
  - Because a user use other services and make private data travel in the network
- Skilled users ("hackers") can:
  - Enter the door and violate the host
    - (privacy, destruction or denial of services)
  - Violate the network connection ("Sniffing" the data travelling in the network by going in and out the gate)
  - Pretend to be who they are not (another host, another user)

Attacks Examples (1)

- Denial of Service
  - Send continuosly packets to the same host (network congestion)
  - Send millions of mail to the same server (HD congestion)
  - Invoke continously heavy services
- Sniffing
  - Special program are capable of reading the data that travels in the network (e.g., can read passwords and private E-mails)

Attacks Examples (2)

- Given a passwd (having sniffed it)
  - Access services that require a password (e.g., E-mail, telnet, ftp)
  - Retrieve private data, destroy private data
- Getting control of the computer
  - Possibly exploiting pitfalls of the operating system (e.g., “sasser”) or sever inadvertedly installed (e.g. telnet
  - Simply disturbing the user
  - Or exploiting the computer resources for illegal purposed (e.g., storing of pornogrphic material)

Modern Security Attacks

- Are based on problems (“bugs”) of Web and Internet applications
  - Outlook express automatically can executes attached programs →
    - viruses sent by E-mails
    - Troyan horses installed on clients
  - Browsers execute Java applets on clients and can send data to servers →
    - “hacked” applets can destroy or read data on clients
    - “hacked” browsers can send “troyan horses” to servers

Protecting a Network

- For local services
  - Make sure that only specific user (possibly from specific hosts) can use services
  - Gurantee identification and authorization in the network
  - Check the services of the operating systems and their integrity
- For travelling data
  - Make sure that the data you send/receive can be read only by authorized users (**Cryptography**!)
- A network should be protected both from external (people accessing the network from outside it) and internal (people using the computers of the network) attacks!

Traditional Unix Security (BSD)

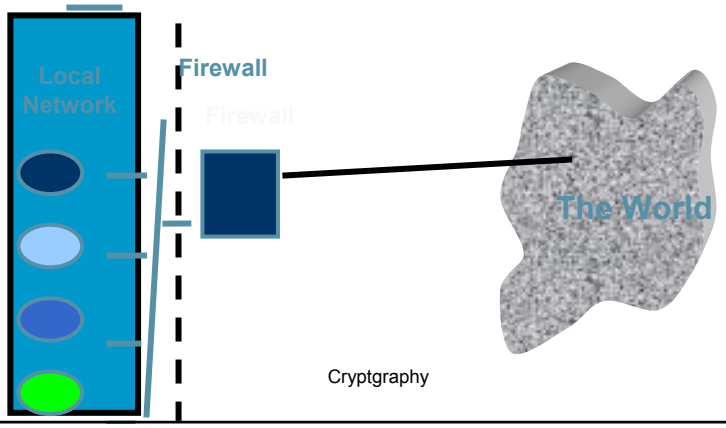
- Based on option 2 – trust users on trusted hosts.
  - if the user has been authenticated by a trusted host, we will trust the user.
  - Then, the user can use the services on a machine (or, which is the same, its programs can access the servers)
- Advantages: very simple
- Problems:
  - In other words, the whole networks rely on the fact that internal computers (and internal users) are trusted. Does not deal with Internal attacks.
  - Authentication of hosts based on IP address! (doesn't deal with IP spoofing). External attacks can come by having a computer pretend to be a computer of the network
- Better solutions are needed!

Firewall-like Solutions

- To protect the network from the external
- Firewall: a network component that separates two networks and (typically) operates in the upper layers of the OSI reference model (Application layer).
- Screening Router: a discriminating router that filters packets based on network layer (and sometimes transport layer) protocols and addresses.

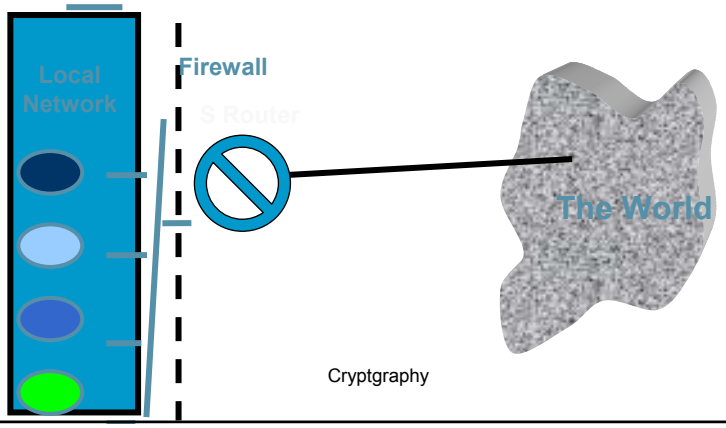
Firewall

- Firewall: all data and requests crosses the firewall, which can
  - Make cross only specific protocols
  - Request one-time password to access services



Screening Router

- Similar to a firewall
  - Can select which datagram to route
  - More drastic solution
  - Less flexible



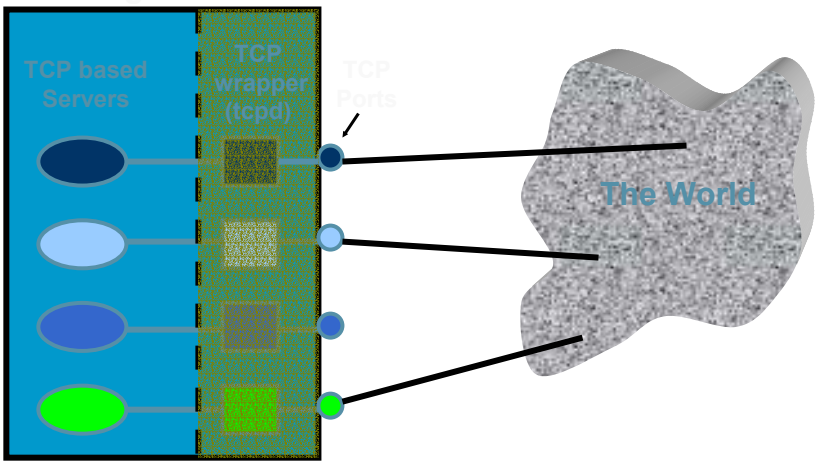
An alternative: Private IP address

- To protect a network from the external
- Define an Intranet
  - No public IP address (i.e., IP addresses valid only in the internals of the network and not recognized outside)
- Make only the IP of a server public
  - The server is the only computer actually visible from the external world
  - Only the server can have public services
- Advantages: very secure
- Problems
  - limits the capabilities of the computers (cannot install servers or do things like chatting, Netmeetings, etc.)
  - Discouraged by Internet community

TCP Wrapper (Software Firewall)

- To protect a single computer from external and internal
- TCP wrapper is a software program that provides some firewall-like functionality.
  - A single host (really just a few services) is virtually isolated from the rest of the world via the TCP wrapper which itself runs.
  - Functionality includes logging of each and every network requests for service and access control.
- Configuration:
  - The configuration files for tcpd specify which hosts are allowed/denied which services.
  - Entire *domains* or IP networks can be permitted or denied easily.
- Advantages: very secure if operating system secure (XP? Ah, ah, ah!!)
- Problem: very costly to configure and maintain (each and every computer should be configures)

TCP Wrapper Picture



TCP Wrapper Example: `tcpd`

- In UNIX Systems
  - The `tcpd` daemon checks out incoming TCP connections for all servers, before the real destinations server gets the connection.
  - Can find out source IP address and port number.
  - A log message can be generated indicating the service name, client address and time of connection.
    - It is important to know what happens and what happened
- `tcpd` can use client addresses to authorize each service request.

Centralized Security Solutions

- To protect the network from the internal
  - In addition to a Firewall protecting it form the external
- Install a server
  - Make this server act as a centralized controller for each and every network request
  - Who can access the PCs of the network
  - Who can use which services
- Advantages: easy to be installed and maintained
- Problem:
  - low performances (bottleneck)
  - The correct configuration of the server is critical

Example: the Info Lab

- A firewall protecting from external attacks
- A Linux servers to authenticate users
  - Any access to a PC goes to the server in a crypted way
  - The server checks the data (username and passwd) before granting access
  - Only authorized users in the data base can access the PC
- For each and every network request (printing, surfing, etc.)
  - The server checks if the user has authorization to access the service

Security in Java (1)

- Java enables to specify what classes (i.e., what the objects of this classes) can and cannot do
  - This is very important since classes can also be loaded at run time from the network
- To do this:
  - Define a policy file (see next slide) to specify what classes can and cannot do
  - Make the *main* of your application create a security manager object

    ```
    System.setSecurityManager(new SecurityManager());
    ```
  - OR create the security manager by the command line

    ```
    Java –Djava.security.manager
    ```
  - Run applications with the security manager policy option, specifying which policy file to refer to

    ```
    Java –Djava.security.policy=mypolicyfile MyApplication
    ```

Security in Java (2)

- Examples of entries in a policy file
- Grant sockets connection to a specific server only

```
Grant java.net.Socket permission
Java.net.SocketPermission "155.185.2.13", "accept, connect"
```

- Grant objects of the Professor class to read the file StudentsMark.txt

```
Grant univ.Professor permission
Java.io.FilePermissions "StudentMark.txt", "read"
```


Security in Java (3)

- Once a class executes, it can go on without problem but…
  - If it tries to do something it is not granted to a SecurityException is raised
- An object can check whther it has permission or not before doing an action:

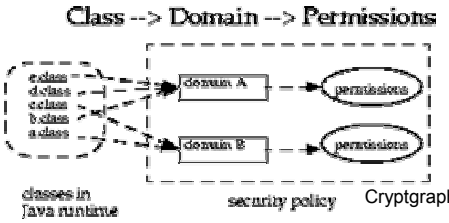
```
// retrieve a reference to the security manager
SecurityManager sec = System.getSecurityManager();

// create a specific permission object
FilePermission ff = new FilePermission("Stud.txt, "read")

// ask the security manager if the permission is granted
If(sec.checkPermission(ff))…
```

Security in Java (4)

- Security Domains
  - It is possible to group in different sets different types of permissions
  - These defines differerent “security domains”
- Then:
  - It is possible to specify for a class to which security domains it belongs to
- This approach makes the definition of the security policies very flexible and modular
- The example of the applets:
  - All applets are in the so called “sandbox” security domain, where they have almost no rigths to do anything



Cryptography

Protecting Travelling Data

- One can control access to a host or to a network
  - Require password
  - Require known hosts and clients
- Still:
  - As we make data and password travel in the network
  - We must avoid sniffing these data and enable reading them

Terminology

- Encryption: Scramble data so that only someone with a secret can make sense of the data.
  - For data that travels in the Internet
- Decryption: Descrambling encrypted data.
- DES: Data Encryption Standard: secret key cryptographic function standardized by NBS (NIST).

Terminology (cont.)

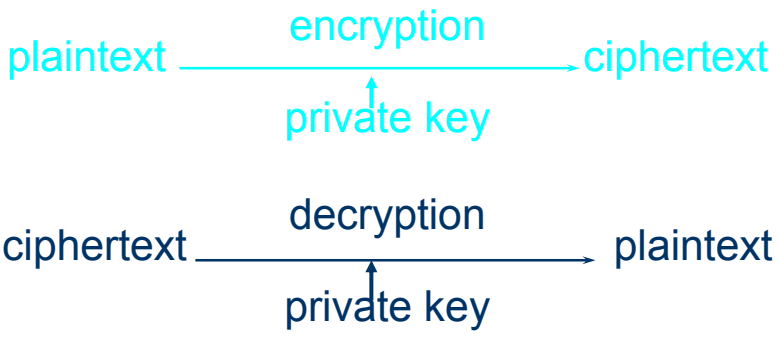
- Secret Key Cryptography: a cryptographic scheme where the same key is used to encrypt and decrypt.
- Public Key Cryptography: a cryptographic scheme where different keys are used for encryption and decryption.

Secret Key Cryptography

- Single key used to encrypt and decrypt.
- Key must be known by both parties.
- Assuming we live in a hostile environment (otherwise - why the need for cryptography?), it may be hard to share a secret key.

Using Private Keys

- Private keys are used for both encrypting and decrypting.



Private Key Cryptography: Caesar's Code & Variations

- Sending "How are you"
  - K =3
  - H+3 =L, o+3=R, etc. etc.
  - Crypted message: "Lrz duh brw"
- Trivial to decrypt without the key
  - Requires at most 26 attempts…

Private Key Cryptography: Example of Algorithm (1)

- Modern version of the Caesar's code
- Rely on the bit representation of a message
- Sending "How are you"
  - In ASCII: 45 51 59 39 53 50 51 54
  - Group into n-bit numbers, e.g., 455159, 395350, etc.
  - Private Key (e.g. a number of n-bit): k=115454
  - Crypted message: (455159*k%999999), (395350*k%999999), etc… = 525253864562339005829477 16
  - Requires about 999999/2 attempts to decrypt

Private Key Cryptography: Example of an Algorithm (2)

- Receiving the message
  - Crypted: 525253864562339005829477 16
  - Decomposing: (525253*999999)/k=455159, etc. etc.
  - Reconstructing: 45 = "H", 51="o". Etc, etc,
  - Eventually: "How are you"
- Extensions to the Caesar code:
  - Vigenere's code: apply different key to different parts of the message
  - Apply the Caesar code several times, by permutating according to given rules the content of the message

Private Key Cryptography: DES: DataEncryption Standard

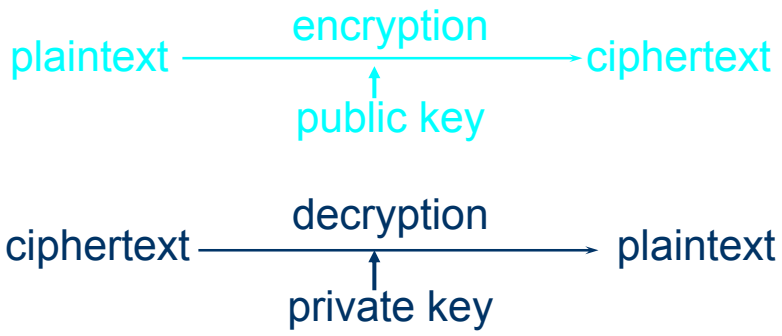
- International standard (1993)
  - 48-bit key (exploiting 32 bits at a time of the key)
  - Applied to groups of 64 bit in the message
- How does it work:
  - Take bits 0-63
  - Permute bits 0-31 and 32-63
  - Apply a key composed of the first 32 bits of the main key to each of the two 32-bit groups
  - Do the same for bits 63-127, and so on….
- Permute bits 32-64 and 64-95
  - Apply key
- Repeat 16 times….by applying the bits 1-32, 2-33, 3-34, of the main key

Public Key Cryptography (a.k.a. asymmetric cryptography)

- Relatively new field - 1975 (as far as we know, the NSA is not talking).
- Each entity has 2 keys:
  - private key (a secret)
  - public key (well known).

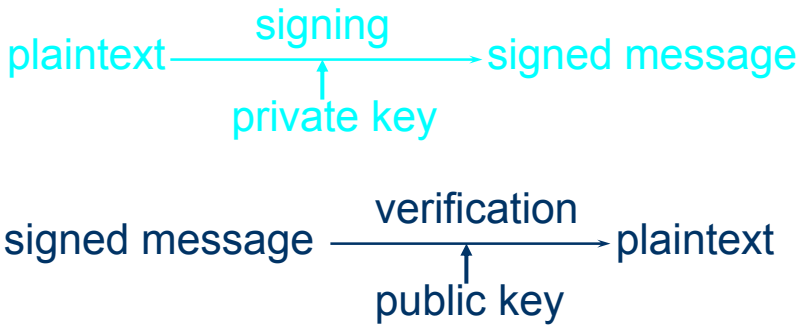
Using Keys

- Public keys are used for encrypting.
- Private keys are used for decrypting.



Digital Signature

- Public key cryptography is also used to provide digital signatures.



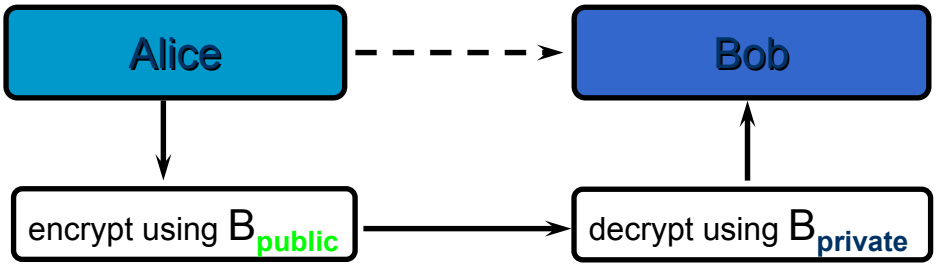
Transmitting over an insecure channel.

Alice wants to send Bob a private message.

A_{public} is Alice's public key.
$A_{private}$ is Alice's private key.
B_{public} is Bob's public key.
$B_{private}$ is Bob's private key.

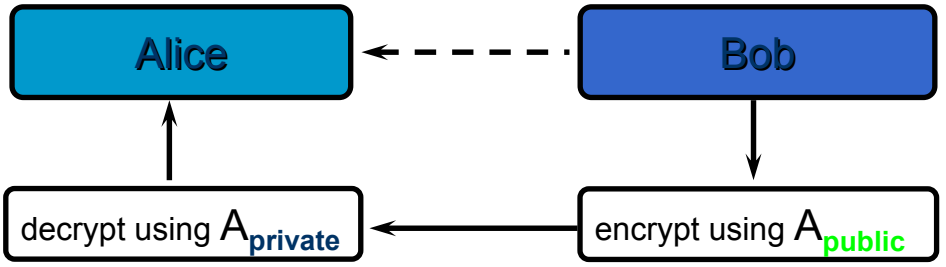


Hello Bob,
Wanna get together?





OK Alice, Your place or mine?



RSA: Example of a public key algorithm

- RSA
  - Developed by R. Rivest, L. Adleman, A. Shamir, MIT
- Based on a simple property:
  - It is very difficult (NP problem) to factorize a very big number into its prime factors
  - Example: 3980301621=23447*32243
  - Impossible to be determined (in reasonable times) even with the most powerful computers…
- Applied on the bit representaton of a message

RSA: keys

- Key idea:
  - Choose any three big prime numbers at random: *E,p,q*
  - Compute *n=p*q*
  - Make the couple *(E,n)* the public key
  - Exploit your private numbers *p,q* to generate a private key with which you can decipher messages

RSA: crypting

- Sending a message:
  - Transform it into a set of numerical sequences (as the private key cryptography scheme): P_1, P_2, ..P_i, etc. (make sure $P_i < N$ for any i
  - Compute: $C_i=(P_i^E)\%n$ for all i
  - Send all the C_i as the crypted message
  - Impossible to decrypt even knowing *E* and *n*!!
    - The % operation is irreversible, and makes it generally impossible to reconstruct the original operators…BUT
    - We could do that only if *n* is the product of two primes and if we know what are these two primes!!!

RSA: decrypting

- Receiving a message:
  - Compute: $f(n)=(p-1)(q-1)$ Euler Function
  - Requires knowing p and q
  - Then, the properties of the Euler function ensures that, for any number E, there exists a number D such that: $(D*E)\%fi(n)=1$
  - Compute $D = (E^{-1})\%fi(n)$
  - D is the real private key!

  - Receive crypted message: $C_1, C_2, \ldots C_i, \ldots$
  - Compute: $P_i=(C_i^D)\%n$ for all i

RSA: Explanation

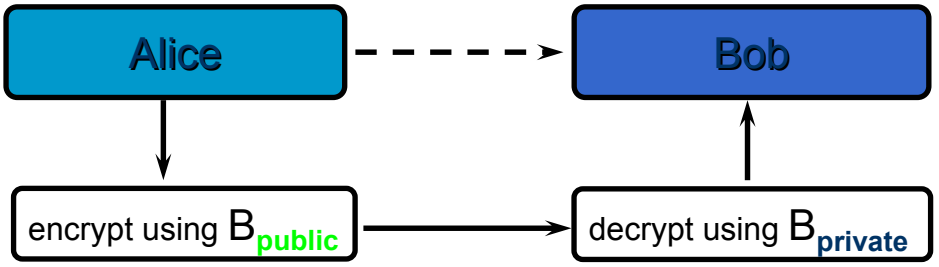
- The properties of % operation ensures that:
  - In an expression which is subject to a $\%n$ operation, the exponents of the expression can be reduced by $\%fi(n)$, i.e.:

    $$(x^y)\%n=(x^{y\%fi(n)})\%n$$

- *Given the above property:*
  - $(C_i^D)\%n=(P_i^E\%n)^D\%n=P_i^{ED}\%n=P_i^{ED\%fi(n)}\%n$
  - *Since* $(E*D)\%fi(n)=1$ *then* $P_i^{ED\%fi(n)}\%n=P_i\%n$
  - *Since* $E > P_i$ *for any I then* $P_i\%n=P_i$



Bob's dilemma



Is the message really from Alice?

Bob's Dilemma

- Nobody can read the message from Alice, but anyone could produce it.
- How does Bob know that the message was really sent from Alice?

- Bob may be comforted to know that only Alice can read his reply
- But this is not enough…

Solution: Alice can sign her message!

- Alice can create a digital signature and prove she sent the message (or someone with knowledge of her private key).
- The signature can be a message digest encrypted with $A_{private}$. (only Alice could encrypt a message in that way – so it is a secure identifier of Alice)
- How can it be decrypted?

Solution!: Symmetry of keys

- Requires a cryptography algorithms in which public and private keys are interchangeable: the message can be:
  - crypted using public and decrypted using private
  - Viceversa: crypter using private and decrypted using public
- Most algorithms (as RSA) allow this:
  - $C_i=(P_i^E)\%n$
  - $P_i=(C_i^D)\%n$
  - Encryption/decryption symmetric for keys *D* and *E*
- So:
  - If Bob can decipher a message with Alice's public key
  - This must have been produced with Alice's private key

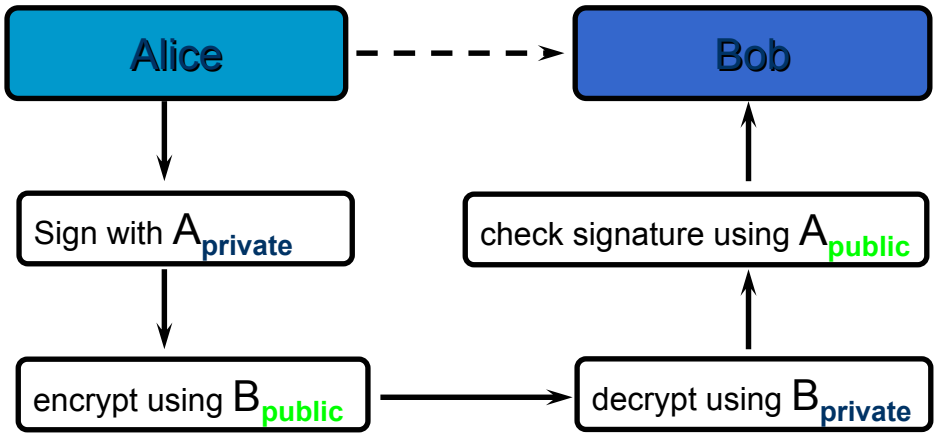
Message Digest as Digital Signatures

- Also known as “hash function” or “one-way transformation”.
- Transforms a message of any length and computes a fixed length string
  - Example: (*34509823452342 4%1024*)
- We want it to be hard to guess what the message was given only the digest.
  - Guessing is always possible but hard...

Alice’s Signature

- Alice integrates her original message with a digest of the message, and encrypts the message digest with $A_{private}$.
- Bob can decrypt the message digest using A_{public}.
- Bob can compute the message digest himself (once he has decrypted the original message).
- If the 2 message digests are identical, Bob knows Alice (and only her) sent the message.

Revised Scheme



Why the digest?

- Alice could just encrypt her name, and then Bob could decrypt it with A_{public}.
- Why wouldn't this be sufficient?
- Suppose Alice denies she sent the message?
- Bob can *prove* that only someone with Alice's key could have produced the message

A Possible Problem with the Public Key

- Suppose Bill receives a message from Monica *including* a digital signature.
- Bill sends the same message to Al so that it looks like the message came from Monica.
- Bill includes the digital signature from the message Monica sent to him.
- Al is convinced Monica sent the message!

Solution?

- Always start your messages with:
  - Dear Bill,
- Create a digest from the encrypted message and sign that digest.
- There are many other schemes as well.

Speed

- Secret key encryption/decryption algorithms are much faster than public key algorithms.
- Many times a combination is used:
  - use public key cryptography to share a secret key.
  - use the secret key to encrypt the bulk of the communication.

The Issue of Certification

- Who Generates and distributes Keys?
- For symmetric keys (DES)
  - KDC, Key distribution center
  - Takes care of generating and privately distributing to those who needs it a secret key
- For public keys:
  - Certification Authority
  - Ensures that a public key belong to a specific person!
  - Why is this needed?

Why Certification?

- Assume I want to send a message to Alice
  - I must somehow know its public key pk_{alice}
  - I can read that somewhere, e.g., on Alice's web site or on the Yellow pages
- However:
  - It is possible that Charles somehow has maliciously made me think $pk_{charles}$ was the key of Alice
  - Then I send a message thinking only Alice can read it, and instead only Charles can read
- I must be really sure about which is the public key of Alice!
  - CERTIFICATION OF PUBLIC KEY!

Certification Authority

- Centers for generation and distribution of private and public keys
  - Must be recognised the the IANA (Internet Administration Network Authority)
  - In Italy: Politecnico di Torino
- For programs exploiting cryptography in message excange (e.g., https)
  - Before sending a crypted message (e.g., a credit card number)
  - The public key is retrieved
  - The certification authority is asked if the public key really belongs to the claimed entity
  - Explorer and Netscape alert the user whenever a public key is used that is not recognized by the Certification Authority

Cryptography in Java

- package javax.crypto.spec
- Information at:
  - http://java.sun.com/j2se/1.4.2/docs/guide/security/CryptoSpec.html
- A large number of classes to:
  - Encrypt messages (DES, RSA)
  - Generate messages digests (HASH functions)
  - Generate certificates

Secure Protocols

- There are a growing number of applications for secure protocols:
  - email
  - electronic commerce
  - electronic voting
  - homework submission

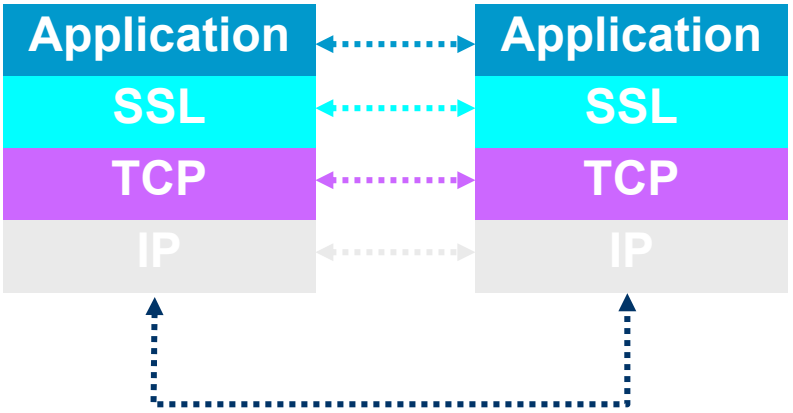
Secure Protocols

- Many application protocols include the use of cryptography as part of the application level protocol.
  - The cryptographic scheme employed is part of the protocol.
  - If stronger cryptographic tools become available we need to change the protocol.
  - NOT GOOD!

SSL and TLS

- Secure Sockets Layer (SSL) is a different approach - a new layer is added that provides a secure channel over a TCP only link.
- TLS is Transport Layer Security (IETF standard based on SSL).

SSL layer



Advantages of SSL/TLS

- Independent of application layer
- Includes support for negotiated encryption techniques.
  - easy to add new techniques.
- Possible to switch encryption algorithms in the middle of a session.

What SSL Does

- Opening a connection:
  - Handshake to establish a cryptography protocol (and agree on the Key)
  - The server authenticates the client (or viceversa)
- Exchanging messages:
  - All messages are crypted according to the established protocol

SSL in Java: SSLSocket

- Classes: **SSLSocket**, **SSLServerSocket**
- Work like just any socket **BUT**:
  - Must be created by using a special object of the class **SSLServerSocketFactory** (acts as a centralized manager for sockets)
  - Requires that before running the program the public and private keys to be used are stored somewhere and made available to the program: the **keytool** program serve this purpose

HTTPS

- HTTPS is HTTP running over SSL.
  - used for most secure web transactions.
  - HTTPS server usually runs on port 443.
  - Include notion of verification of server via a certificate.
  - Central trusted source of certificates.